

Hospital Management System

Data Structures Final Project Report (CLO-3)

Course: Data Structures
Class: BSCS III-B (Spring 2025)
Instructor: Dr. Sumaira Farid
Submission Date: 16/12/2025

1. Project Title

Hospital Management System using Data Structures & Algorithms in C++

Group Members

- **Hammad Mehmood** (241795)
 - **Muhammad Abdullah Khan** (241921)
 - **Muhammad Zunnorain Haider** (241918)
-

2. Introduction & Real-World Problem Background

Modern hospitals deal with a large volume of patients, doctors, appointments, and emergency cases on a daily basis. Managing this information manually or with poorly designed systems leads to inefficiency, delays in emergency treatment, and poor decision-making. Hospitals also require fast record lookup, priority-based patient handling, relationship modeling between departments, and analytical insights for management decisions.

This project implements a **console-based Hospital Management System** that simulates real-world hospital operations using **multiple data structures**. The system efficiently manages patient records, doctor information, appointment scheduling, emergency handling, departmental networks, and system analytics. The project emphasizes **data-driven decision making, algorithmic efficiency, modular design, and practical application of core and advanced data structures**, fulfilling CLO-3 requirements.

3. Objectives

The main objectives of this project are:

- To design a modular, menu-driven hospital management system in C++ using OOP principles
 - To integrate multiple data structures to handle different system modules efficiently
 - To demonstrate insert, delete, search, and traversal operations
 - To apply advanced data structures such as **Hash Tables, Graphs, Priority Queues, and Trees**
 - To support file handling for persistent data storage
 - To generate analytical reports for better hospital decision-making
-

4. System Constraints & Assumptions

- Console-based system (no GUI)
 - Assumes correct user input format where specified
 - System is designed to handle **100–500+ records efficiently**
 - Departments and graph connections are pre-initialized for demonstration
 - File storage is text-based for simplicity and portability
-

5. Overall System Architecture

The system follows an **object-oriented and modular architecture**, where each real-world entity and system module is mapped to a suitable data structure:

- **Patient Management** → Hash Table
- **Doctor Management** → Binary Search Tree
- **Appointment History** → Doubly Linked List
- **Emergency Handling** → Priority Queue (Heap)
- **Department Network** → Graph (Adjacency List)
- **Activity Tracking** → Stack

The `HospitalManagementSystem` class acts as a controller that integrates all modules and provides a unified menu-driven interface.

6. Data Structures Used & Justification

6.1 Hash Table (Patient Records)

- **Purpose:** Fast lookup of patient records by Patient ID
- **Implementation:** Chaining using vector lists
- **Operations Demonstrated:** Insert, Search, Delete, Traversal
- **Time Complexity:**
 - Average case: $O(1)$
 - Worst case (collision): $O(n)$

Justification: Hospitals require instant access to patient data; hash tables provide efficient retrieval.

6.2 Binary Search Tree (Doctor Records)

- **Purpose:** Organized storage of doctors by Doctor ID
- **Operations:** Insert, Search, Delete, Inorder Traversal
- **Traversal Output:** Displays doctors in sorted order
- **Time Complexity:**
 - Average: $O(\log n)$
 - Worst case: $O(n)$

Justification: BST maintains hierarchical and sorted data, suitable for staff management.

6.3 Doubly Linked List (Appointment Management)

- **Purpose:** Maintain appointment history
- **Operations:** Insert at end, delete by ID, traversal
- **Advantage:** Easy forward/backward traversal and deletion
- **Time Complexity:** $O(n)$

Justification: Appointment history requires sequential access and flexible deletion.

6.4 Priority Queue / Heap (Emergency Queue)

- **Purpose:** Handle emergency patients based on severity
- **Ordering Rule:** Higher severity treated first
- **Operations:** Enqueue, Dequeue, Display
- **Time Complexity:** $O(\log n)$

Justification: Critical patients must be prioritized—heap-based scheduling simulates real emergency rooms.

6.5 Graph – Adjacency List (Department Network)

- **Purpose:** Represent hospital department connections
- **Algorithms Implemented:**
 - Breadth-First Search (BFS)
 - Depth-First Search (DFS)
 - Dijkstra's Shortest Path Algorithm

Justification: Graphs model real-world hospital layouts and referral paths effectively.

6.6 Stack (Activity Log)

- **Purpose:** Maintain system activity history
- **Operations:** Push, Display, File Save
- **Nature:** Last-In-First-Out (LIFO)

Justification: Ideal for logging recent system actions and undo-like behavior.

7. Mandatory System Modules Mapping

Module	Requirement	Data Structure Used
Core Record Management	Patient & Doctor records	Hash Table, BST
Task & Process Handling	Emergency processing	Priority Queue
Hierarchical Storage	Sorted doctors	BST
Smart Search & Analytics	Reports & statistics	Hash Map, Vector
Relationship/Network	Department connections	Graph
Log & Activity Tracking	System logs	Stack

8. File Handling & Persistence

The system supports **permanent storage** using text files:

- `patients.txt`
- `doctors.txt`
- `appointments.txt`
- `activity_log.txt`

Features:

- Automatic data loading on system start
 - Data saving on exit or manual save
 - Recovery of previous sessions
-

9. Analytics & Reporting Module

The analytics module generates:

- Total patients, doctors, appointments
- Emergency queue statistics
- Disease frequency analysis
- Doctor specialization distribution

These insights help administrators understand trends and resource allocation.

10. Demonstration Features

During live demonstration, the system shows:

- Menu-driven navigation
 - Insert, search, delete, traversal operations
 - Emergency patient prioritization
 - Graph traversal (BFS, DFS)
 - Shortest path calculation using Dijkstra
 - File save/load functionality
 - Analytical report generation
-

11. Conclusion

The Hospital Management System successfully integrates **multiple core and advanced data structures** into a single, real-world application. The project demonstrates efficient data handling, modular programming, and algorithmic thinking. It fulfills all CLO-3 requirements and showcases how data structures solve practical problems in healthcare systems.

12. Future Improvements

- GUI-based interface using frameworks
 - Database integration (SQL/NoSQL)
 - Authentication and role-based access
 - Bed management and ICU capacity tracking
 - Advanced recommendation systems using graphs
-